

نظام إدارة العمل عن بعد " لشركة برمجة "

Remote work management system

"For a Programming company"

مشروع هندسة برمجيات

إعداد

تالا الحكيم

الإطار النظري

1.1 المقدمة :

في عصر التكنولوجيا الحديثة , أصبح العمل عن بعد منتشر في الشركات والمؤسسات وبشكل كبير في مختلف المجالات ولا سيما في مجال الهندسة والبرمجة.

يعتبر نظام إدارة العمل عن بعد نظام مهم يساعد الشركات والمؤسسات على تحسين جودة العمل وتقليل التكاليف , يتيح هذا النظام إنشاء بيئة عمل عن بعد ومراقبة وتقييم أداء المبرمجين (دون اختراق الخصوصية) , وتقديم ملاحظات وتقارير دورية ويومية بالإضافة إلى حفظ برمجيات ومشاريع الشركة والعاملين فيها ومعلوماتهم ضمن قاعدة بيانات.

2.1 أهداف المشروع :

يهدف هذا المشروع إلى دراسة وتطبيق نظام إدارة العمل عن بعد في بيئة عمل معينة وهي شركة برمجيات مع التركيز على الفوائد التي يمكن أن يجلبها هذا النظام.

سيتم تحليل كيفية تحسين كفاءة المطورين و تقليل التكاليف وزيادة الإنتاجية للشركة من خلال استخدام النظام (كما أنه يقدم ميزات تساعد في تطوير أداء أعضاء الشركة وتعزيز سهولة الوصول فيما بينهم أثناء العمل لتقديم أفضل أداء ممكن من فريق العمل)

3.1 نبذة تعريفية عن نظام إدارة العمل عن بعد

هو نظام يعمل بشكل أساسي على مراقبة العمل وتوفير التواصل بين أعضاء فريق العمل بالإضافة إلى إدارة وتنظيم عمل الشركة عن بعد ضمن إطار ممنهج ومرتب حسب مراجع حديثة ومختلفة.

4.1 دراسة لوظائف النظام الأساسية

- **مراقبة العمل :** تتبع النشاط (وقت العمل الفعلي للمبرمجين) , تسجيل وقت تسجيل الدخول والخروج لتحديد وحساب ساعات العمل , بالإضافة إلى استخدام تقنيات كشف الغش (مراقبة النشاط على لوحة المفاتيح مع مراعاة الخصوصية).
- **التواصل بين أعضاء الفريق :** دردشة داخلية (مراسلة فورية بين أعضاء الفريق) , إضافة تعليقات على المهام والمشاريع ومشاركتها بين أعضاء الفريق.
- **إدارة وتنظيم العمل في الشركة :** وذلك من خلال إنشاء نظام إفتراضي يحاكي بيئة العمل داخل الشركة بالإضافة إلى بعض الخصائص الإضافية التي تزيد من تطور العمل.
- **إدارة تكليف المهام والمشاريع :** يتيح هذا النظام آلية عمل سهلة وسريعة لمساعدة المدراء بتكليف المشاريع إلى قائدي الفرق الذين بدورهم يوزعون على شكل مهام إلى أعضاء الفريق وعند الإنتهاء تسليم المهام وتجميعها عند قائدي الفرق وعند الإنتهاء من المشروع يتم تسليمه إلى المدراء.
- **أدوات مساعدة للشركة وكادر العمل :** إضافة أدوات تساعد المدراء والفرق في إنجاز المهام بشكل أسرع وتساعد في تحسين كفاءة البرمجيات مثل (أدوات : تقييم برمجيات وكشف الأخطاء , إقتراحات لحل المشاكل في البرمجة والتصميم , مساعدة في العثور على برمجيات مشابهة للمشروع الجديد من المشاريع القديمة...)

- **إدارة الأرشفة :** حفظ البرمجيات المنجزة وفهرستها ضمن قاعدة بيانات خاصة بالشركة من خلال إعطاء كل برنامج رقم تسلسلي فريد لتسهيل الوصول لمعلوماته كاملة والوصول إليه , إضافة لحفظ معلومات الموظفين كافة في الشركة وتسهيل الوصول لمعلوماتهم كافة (العملية, الشخصية, المشاريع , المهام...)
- **حساب التقييم :** من خلال حساب المتوسط الحسابي لنقاط كل المهمات المنجزة للعاملين في الشركة , وحساب التقييم الخاص من الشركة بالبرمجية من خلال إيجاد المتوسط الحسابي لكل المهمات التي تتبع للمشروع.

5.1 سيناريو عمل النظام :

السيناريو الرئيسي: إدارة مشروع برمجي من البداية إلى التسليم
الأدوار الرئيسية:

. المدير التنفيذي

. قادة الفرق (التطوير، الجودة، الإنتاج...)

. أعضاء الفرق

. النظام(System)

1.1 بدء المشروع(Project Initiation)

الحدث :وصول مشروع برمجي جديد إلى المدير العام.
التفاعلات:

1. المدير يُنشئ المشروع ويقوم "بطلب تكليف مشروع" عبر النظام.
2. النظام يُرسل الطلب تلقائيًا إلى قادة الفرق.

2. تقييم المهام وقبول/رفض المشروع (Task Assessment)

الحدث: قائد الفريق يستلم المشروع.
التفاعلات:

1. النظام يُظهر لقائد الفريق:

- المتطلبات المطلوبة (مهارات، موارد، وقت).
- الموارد المتوفرة و الغير المتوفرة.

2. قائد الفريق يتخذ قرارًا:

- القبول: يبدأ توزيع المهام على أعضاء الفريق.
- الرفض: يُدخل سبب الرفض (مثل: نقص الموارد) عبر النظام.
- النظام يُعيد الطلب إلى المدير مع سبب الرفض.
- المدير يتخذ إجراءً (تأمين موارد، إعادة توجيه المشروع، تكليف فرق أخرى...).

3. توزيع المهام على الفريق (Task Allocation)

الحدث: قبول المشروع من قائد الفريق.
التفاعلات:

1. قائد الفريق يستخدم النظام لـ:

- تقسيم المشروع إلى مهام فرعية.
- تعيين كل مهمة إلى عضو فريق معين (مثال: مُبرمج، مصمم...).

2. النظام يُرسل إشعارات للموظفين مع:

- تفاصيل المهام.
- الجدول الزمني.
- الأدوات المطلوبة (مثل: وصول إلى قاعدة بيانات معينة, برمجية سابقة...).

4. التنفيذ والمتابعة (Execution & Monitoring)

الحدث: أعضاء الفريق يعملون على المهام.
التفاعلات:

1. أعضاء الفريق يُحدّثون حالة المهام عبر النظام (مثال: "قيد التقدم"، "مكتملة", "معلقة"...).

2. النظام يقوم تلقائيًا بـ:

- تسجيل التقدم الزمني للمشروع.
- مراقبة ساعات العمل (مراقبة الدوام).
- إرسال تنبيهات عند تأخر المهام.

3. قائد الفريق يُراقب التقدم عبر لوحة تحكم (Dashboard) ويُدخل ملاحظات للموظفين.

5. مرحلة الجودة والتكامل (Quality Assurance & Integration)

الحدث: اكتمال مهام التطوير.
التفاعلات:

1. قائد فريق التطوير يُرسل المشروع عبر النظام إلى فريق الجودة.
2. فريق الجودة يختبر البرمجية ويُدخل تقارير الأخطاء في النظام.
3. النظام يُرسل التقارير تلقائيًا لفريق التطوير لإصلاح الأخطاء.
4. بعد التصحيح، فريق الإنتاج يستلم المشروع عبر النظام لتعبئته.

6. التسليم النهائي (Final Delivery)

الحدث: اكتمال المشروع.
التفاعلات:

1. قائد فريق الإنتاج يُرسل النسخة النهائية عبر النظام إلى المدير.
2. المدير يراجع المشروع ويوافق على تسليمه للعميل.
3. يُنشئ نسخة أرشيفية في قاعدة البيانات مع:

- سجل المهام.
- تقارير الجودة.
- تقييمات الأداء.

◦ البرمجية النهائية المكتملة

7. الوظائف الإضافية للنظام (System Features)

1. التقييمات الذكية:

- يُحلل النظام أداء الموظفين بناءً على سرعة إنجاز المهام وجودتها.
- يُولد تقارير تقييم تلقائية للترقيات أو التدريب.

2. إدارة الجداول الزمنية:

- يُنشئ جداول زمنية ديناميكية بناءً على أولوية المهام وتوفر الموارد.

3. الأرشفة والبحث:

- جميع البيانات (مشاريع، موظفين، أخطاء) مُخزنة في قاعدة بيانات مرتبة.
- البحث السريع باستخدام فلاتر (مثال: "مشاريع 2023"، "برمجيات معدلة",.....).

4. الدعم الفني التلقائي:

- يُوفر النظام أدوات مساعدة مثل:
 - اتصال مباشر مع الدعم.
 - وصول إلى مكتبة برمجية (Code Library ,Get hub) للاستفادة من الحلول السابقة.

6.1 متطلبات النظام الوظيفية

يتطلب تنفيذ نظام إدارة العمل عن بعد مجموعة من المتطلبات الوظيفية لضمان تحقيق أهدافه ,إلينا بعض المتطلبات الرئيسية التي يجب أن يتضمنها النظام:

● إدارة المهام والمشاريع

- يجب أن يسمح النظام بإنشاء وتوزيع المهام على أعضاء الفريق مع تحديد الأولويات والمواعيد النهائية.
- يجب أن يدعم تتبع حالة المهام (مكتمل، قيد التنفيذ، متأخر).

● التواصل والتعاون

- يجب أن يوفر النظام غرف دردشة ومنتديات مناقشة لكل مشروع.
- يجب أن يدعم مشاركة الملفات (مثل الأكواد، المستندات،الواجهات) مع إمكانية التعديل المشترك في الوقت الفعلي.

● إدارة الوقت والإنتاجية

- يجب أن يتضمن تقارير تلقائية عن ساعات العمل لكل موظف (Time Tracking).
- يجب أن يُنشئ تحليلات عن أداء الفريق (مثل: عدد المهام المكتملة، التأخيرات).

● التكامل مع أدوات المطورين

- يجب أن يتكامل مع منصات مثل **GetHub** و **GetLab** لإدارة الأكواد.
- تزويد المطورين بالأدوات المساعدة للقيام بعملهم والتحسين من كفاءتهم وإنتاجهم وجودة إنتاجهم.

● الأمان والصلاحيات

- يجب أن يُحدد صلاحيات الوصول بناءً على الدور (مثل: مدير، مبرمج، مصمم).
- يجب أن يُشفر البيانات المتبادلة بين المستخدمين (End_to_End Encryption).

● الدعم الفني والتقارير

يجب أن يوفر النظام تقارير دورية بخصوص الأعمال المنجزة والغير منجزة مع الوقت المتبقي لإنجازها إضافة إلى عرض نسبة تقدم المهمة أو المشروع والتقدم المهني للموظف.

7.1 متطلبات النظام غير الوظيفية

هي معايير الجودة والأداء التي تحكم كيفية عمل النظام :

• الأداء (Performance) :

- يجب أن يعمل النظام بسلاسة مع +100 مستخدم متزامن دون تأخير ملحوظ.
- يجب أن تكون زمن تحميل الصفحات أقل من 3 ثوان.

• التوافقية (Compatibility) :

- يجب أن يكون النظام متاحاً على المتصفحات والأجهزة (كومبيوتر, هواتف محمولة)
- وأن يكون نفس الإصدار لدى الجميع.

• الموثوقية (Reliability) :

يجب أن يُنفذ نسخ احتياطي كل 24 ساعة

• الأمان (Security) :

- استخدم المصادقة الثنائية لتسجيل الدخول.
- منع الوصول غير المصرح به عبر جدران الحماية.

• القابلية للتطوير (Scalability) :

يجب أن يدعم النظام زيادة عدد المستخدمين بنسبة 200% دون حاجة لإعادة هيكلة كاملة, وقابل لإضافة كائنات جديدة للنظام دون الحاجة لتغيير جذري.

8.1 المستخدمين الذين يتعامل معهم النظام

هم المستخدمين المستهدفين للعمل عن بعد مثل المبرمجين والمصممين والمنتجين, ومن الممكن قادة الفرق في حال الطوارئ بالإضافة إلى جزء من الكادر الإداري الذي يعمل ضمن مبنى الشركة وعلى النظام معاً. ففي مشروعنا يستهدف النظام الفرق البرمجية وفرق التصميم وفرق الإنتاج وقادة الفرق والمدير التنفيذي ومدير النظام بشكل أساسي.

التحليل والتصميم

1.2 توصيف حالات الإستخدام باستخدام جدول التوصيف القياسي :

جميع المستخدمين All Users

1	رقم حالة الإستخدام :
تسجيل الدخول للنظام	اسم حالة الإستخدام :
جميع المستخدمين	الفاعل الرئيسي :
وجود حساب مضاف أصولاً من مدير النظام	الشروط المسبقة :
تسلسل العمليات	
النظام	المستخدم
	يدخل المستخدم اسم المستخدم وكلمة المرور في واجهة الإدخال
يتحقق النظام من صحة البيانات المدخلة ويطابقها مع البيانات في قاعدة البيانات	
يسجل جميع محاولات تسجيل الدخول صحيحة كانت أم فاشلة , ويحفظ هذه السجلات للمراجعة المستقبلية	
يقوم بتسجيل المستخدم في حالة حضور ويبدأ بحساب ساعات العمل وإضافتها إلى معلومات المستخدم في قاعدة البيانات	
يقوم بمنح المستخدم حق الوصول إلى النظام ويظهر له الواجهة الرئيسية	
	يمنح حق الوصول إلى النظام بعد التحقق من صحة البيانات
يفرض سياسات الأمان المحددة مثل مدة الجلسة وتأمين الحساب بعد عدد معين من محاولات تسجيل الدخول الفاشلة	
يقوم بتنفيذ أداة الذكاء لمراقبة عمل المستخدم (مكافحة الغش..), والأدوات المساعدة الأخرى.	
- فشل في التحقق من بيانات تسجيل الدخول يصدر رسالة خطأ ويوجه المستخدم لإعادة المحاولة أو استرجاع كلمة المرور. - تم تجاوز عدد المحاولات المسموح بها لتسجيل الدخول الفاشل , يؤمن الحساب تلقائياً	التدفق البديل :

ويطلب من المستخدم الإتصال بالدعم (مدير النظام). - فشل في حفظ أو جلب أو تحديث المعلومات في قاعدة البيانات بعد أكثر من محاولتين يظهر إشعار الرجاء التواصل مع الدعم.	
- يتابع مدة الجلسة الزمنية ويقوم بإنهاء الجلسة تلقائيا بعد فترة محددة من عدم النشاط على جهاز الكمبيوتر لضمان الأمان ومكافحة الغش - إدخال المستخدم للصفحة الرئيسية.	التدفق اللاحق :

رقم حالة الاستخدام :	2
اسم حالة الاستخدام :	إرسال ملاحظة
الفاعل الرئيسي :	جميع المستخدمين
الشروط المسبقة :	- وجود حساب مضاف أصولا من مدير النظام - تم تسجيل الدخول بنجاح للنظام - أن يكون محتوى الرسالة المرسله غير فارغ - أن يتم تحديد مستخدم في النظام مرسل إليه
تسلسل العمليات	
المستخدم	النظام
يقوم بالنقر على قسم الملاحظات في القائمة المنسدلة	
	يقوم بتوسيع خيارين جديدين للمستخدم في القائمة المنسدلة : إنشاء ملاحظة , الملاحظات الواردة
يظهر له خيارين جديدين : إنشاء ملاحظة , الملاحظات الواردة	
يقوم بإختيار إنشاء ملاحظة	
	يقوم بتحويله إلى واجهه إدخال الملاحظة
يظهر له واجهه إنشاء الملاحظة	
يقوم بتحديد مستخدم و ثم إدخال معلومات الملاحظة ثم إرسال	
	يقوم النظام بحفظ معلومات الملاحظة وتسجيلها في قاعدة البيانات للمراجعات المستقبلية , ثم يقوم بإرسال الملاحظة للمستخدم المحدد
	يظهر للمستخدم تم إرسال الملاحظة بنجاح

	يظهر للمستخدم إشعار تم إرسال الملاحظة بنجاح
التدفق البديل : - فشل في إرسال الملاحظة يقوم بطلب إعادة إرسالها من المستخدم. - إنتهاء مدة الجلسة يطلب من المستخدم إعادة تسجيل الدخول ثم يقوم بإرسالها. - الرسالة فارغة : يطلب من المستخدم أن يقوم بإدخال معلومات "لا يمكن إرسال ملاحظة فارغة" - في حال نقر المستخدم الرجوع للوراء ينبه المستخدم " سيتم إلغاء الملاحظة " في حال أكمل المستخدم عملية الرجوع للصفحة السابقة لا يتم حفظ الملاحظة. - فشل في حفظ أو جلب أو تحديث المعلومات في قاعدة البيانات بعد أكثر من محاولتين يظهر إشعار الرجاء التواصل مع الدعم.	
- يتابع مدة الجلسة الزمنية ويقوم بإنهاء الجلسة تلقائياً بعد فترة محددة من عدم النشاط على جهاز الكمبيوتر لضمان الأمان ومكافحة الغش. - يتم إرسال الرسالة وإظهار إشعار الإرسال للمستخدم. - إعادة المستخدم للصفة الرئيسية.	التدفق اللاحق :

قادة الفرق و المدراء Admins , Team-Leaders

3	رقم حالة الإستخدام :
إرسال تقييم	اسم حالة الإستخدام :
قادة الفرق و المدراء admins&team-leaders	الفاعل الرئيسي :
- وجود حساب مضاف أصولاً من مدير النظام - تم تسجيل الدخول بنجاح للنظام - أن يكون الحساب (قائد فريق أو مدير) - التقييم يكون بين ال 2-3.8 نقاط	الشروط المسبقة :
تسلسل العمليات	
النظام	المستخدم
	يقوم المستخدم بالتحديد على قسم التقييم
يقوم بنقل المستخدم إلى واجهة التقييم	
	يظهر للمستخدم واجهة التقييم

يقوم بتحديد مستخدم ويقوم بإدخال التقييم ثم إرسال	
يقوم النظام بحفظ عملية التقييم وتسجيلها في قاعدة البيانات	
يظهر للمستخدم تم إرسال التقييم وحفظه بنجاح	
يظهر للمستخدم إشعار تم إرسال التقييم بنجاح	يقوم بحساب التقييم الجديد للمستخدم الذي تم تقييمه ,يظهر للمستخدم الذي تم تقييمه إشعار بالتقييم وتقييمه الجديد ثم تحديث معلومات المستخدم وحفظ التقييم الجديد
يظهر للمستخدم إشعار تم إرسال التقييم بنجاح	
التدفق البديل :	- إنتهاء مدة الجلسة يطلب من المستخدم إعادة تسجيل الدخول ثم يقوم بإرسالها. - فشل في إرسال التقييم يقوم بطلب إعادة إرسالها من المستخدم. - يظهر للمستخدم رسالة "يجب التقييم بين 2-3.8 نقاط" في حال تم التقييم خارج نطاق التقييم. - فشل في حفظ أو جلب أو تحديث المعلومات في قاعدة البيانات بعد أكثر من محاولتين يظهر إشعار الرجاء التواصل مع الدعم.
التدفق اللاحق :	- يتابع مدة الجلسة الزمنية ويقوم بإنهاء الجلسة تلقائياً بعد فترة محددة من عدم النشاط على جهاز الكمبيوتر لضمان الأمان ومكافحة الغش. - يتم عملية التقييم وحفظها وإشعار المستخدم الذي تم تقييمه وعرض تقييمه الجديد. - بعد تخطي المستخدم لإشعار التقييم يقوم بتحويله للصفحة الرئيسية.

قادة الفرق Team_Leaders

رقم حالة الاستخدام :	4
اسم حالة الاستخدام :	إنشاء وإرسال مهمة
الفاعل الرئيسي :	قادة الفرق Team-Leaders
الشروط المسبقة :	- وجود حساب مضاف أصولاً من مدير النظام - تم تسجيل الدخول بنجاح للنظام - أن يكون الحساب (قائد فريق) - أن يكون المستخدم متوفر (ليس في إجازة)

تسلسل العمليات	
المستخدم	النظام
يقوم بالنقر على قسم المهمات في القائمة المنسدلة	
	يقوم بتوسيع خيارات جديدة منسدلة : إنشاء مهمة , المهمات المنجزة , عرض كل المهمات
يظهر خيارات منسدلة جديدة : إنشاء مهمة , المهمات المنجزة , عرض كل المهمات	
يقوم بإختيار إنشاء مهمة	
	يقوم بتحويله إلى واجهة إنشاء مهمة
يتم التحويل إلى واجهة إنشاء مهمة	
يقوم بإدخال تفاصيل ومعلومات المهمة وتحديد المستخدم المنفذ سم يختار إنشاء وإرسال	يتم حفظ المعلومات المدخلة في الذاكرة المؤقتة
	يقوم بحفظ المهمة ومعلوماتها في قاعدة البيانات ثم يقوم بإرسالها إلى المنفذ وإدراجها في قسم المهمات الجديدة لديه
	يقوم بإظهار إشعار تم الإنشاء والإرسال بنجاح
يظهر للمستخدم تم الإنشاء والإرسال بنجاح	
التدفق البديل :	- إنتهاء مدة الجلسة يطلب من المستخدم إعادة تسجيل الدخول ثم يقوم بإرسالها. - فشل في إرسال المهمة يقوم بطلب إعادة الإرسال مرة أخرى - فشل في حفظ أو جلب أو تحديث المعلومات في قاعدة البيانات بعد أكثر من محاولتين يظهر إشعار الرجاء التواصل مع الدعم.
التدفق اللاحق :	- يتابع مدة الجلسة الزمنية ويقوم بإنهاء الجلسة تلقائيا بعد فترة محددة من عدم النشاط على جهاز الكمبيوتر لضمان الأمان ومكافحة الغش. - يتم إرسال المهمة وحفظها في قاعدة البيانات - إدراج المهمة فيقسم المهمات الجديدة لدى المنفذ - إرسال إشعار تنبيه للمنفذ بمهمة جديدة - إشعار المستخدم بنجاح العملية

المطورين Dev-Users

رقم حالة الإستخدام :	5
----------------------	---

تلقى مهمة جديدة	اسم حالة الإستخدام :
المطوّرين Dev-Users	الفاعل الرئيسي :
- وجود حساب مضاف أصولا من مدير النظام - تم تسجيل الدخول بنجاح للنظام - أن يكون الحساب (مطوّر) - وجود مهمات جديدة	الشروط المسبقة :
تسلسل العمليات	
المستخدم	النظام
يقوم بالنقر على قسم المهمات في القائمة المنسدلة	
يظهر خيارات منسدلة جديدة : إنهاء مهمة, المهمات الجديدة, مهماتي	يقوم بتوسيع خيارات جديدة للمستخدم : إنهاء مهمة, المهمات الجديدة, مهماتي وعرضها للمستخدم بعد جلبها من قاعدة البيانات
يقوم بالنقر على المهمات الجديدة	
يظهر قائمة بكافة المهمات الجديدة التي أرسلها القائد	يقوم النظام بجلب جميع المهمات الجديدة التي أرسلت للمستخدم من قاعدة البيانات وعرضها له
يقوم بالنقر على مهمة ما	
تظهر تفاصيل ومعلومات المهمة مع خيار القبول أو الرفض	يقوم بجلب جميع معلومات المهمة وعرضها للمستخدم وتخيره بالقبول أو الرفض
في حال النقر على قبول	
يظهر إشعار تم قبول المهمة بنجاح	يقوم النظام بتحديث حالة المهمة إلى قيد التقدم أو معلقة في حال المستخدم يعمل على مهمة أخرى يقوم بحفظ العملية بقاعدة البيانات يقوم بإرسال تقرير قبول للقائد وإشعاره بقبول المهمة
في حال النقر على الرفض	يقوم بعرض إشعار تم قبول المهمة بنجاح
يتم عرض شاشة إدخال سبب الرفض مع خيار إرسال	
يقوم المستخدم بإدخال سبب الرفض ثم إرسال	

يقوم النظام بتحديث حالة المهمة عند المرسل والمرسل إليه إلى مهمة مرفوضة يقوم بحفظ العملية في قاعدة البيانات يرسل تقرير الرفض للقائد ويتم إشعاره برفض المهمة	
يظهر للمستخدم تم رفض المهمة بنجاح	
يتم عرض تم رفض المهمة بنجاح	التدفق البديل :
- إنتهاء مدة الجلسة يطلب من المستخدم إعادة تسجيل الدخول. - فشل في حفظ أو جلب أو تحديث المعلومات في قاعدة البيانات بعد أكثر من محاولتين يظهر إشعار الرجاء التواصل مع الدعم. - في حال فشل في قبول أو رفض المهمة يطلب من المستخدم إعادة المحاولة.	
- يتابع مدة الجلسة الزمنية ويقوم بإنهاء الجلسة تلقائياً بعد فترة محددة من عدم النشاط على جهاز الكمبيوتر لضمان الأمان ومكافحة الغش. - يقوم بعرض المهمات الجديدة وتفاصيلها وتخيير المطور بقبولها أو رفضها , عند القبول يتم اعتبار المهمة قيد التقدم أو معلقة. - يقوم بحفظ العملية في قاعدة البيانات وتحديث معلومات المهمة. - يقوم بإرسال تقرير القبول/الرفض للقائد وإشعاره بذلك. - في حال عدم وجود مهام جديدة يظهر للمستخدم لا يوجد مهام جديدة.	التدفق اللاحق :

system-Admin مدير النظام

6	رقم حالة الإستخدام :
عرض أو تعديل معلومات مستخدم	اسم حالة الإستخدام :
system-Admin مدير النظام	الفاعل الرئيسي :
- تم تسجيل الدخول بنجاح للنظام - أن يكون الحساب (مدير نظام -system-admin) - تم استلام تقرير تعديل لمستخدم	الشروط المسبقة :
تسلسل العمليات	

المستخدم	النظام
يقوم بالنقر على قسم المستخدمين في القائمة المنسدلة	
	يقوم بتوسيع خيارات منسدلة جديدة للمستخدم : إضافة مستخدم , حذف مستخدم , عرض أو تعديل معلومات المستخدمين
يظهر خيارات منسدلة جديدة : إضافة مستخدم , حذف مستخدم , عرض أو تعديل معلومات المستخدمين	
يقوم بالنقر على عرض أو تعديل معلومات المستخدمين	
	يقوم بعرض قائمة المستخدمين للمستخدم
يتم عرض قائمة المستخدمين	
يقوم بالنقر على مستخدم معين	
	يقوم بتوسيع معلومات المستخدم مع خيار تعديل
يتم عرض معلومات المستخدم المحدد مع خيار تعديل	
في حال أراد التعديل يقوم بالنقر على تعديل	
	يقوم بتحرير خيار الكتابة وتعديل المعلومات للمستخدم مع خيار حفظ
يتم تحرير تعديل معلومات المستخدم مع خيار الحفظ	
يقوم بإدخال المعلومات الجديدة ثم النقر على حفظ	
	يقوم النظام بحفظ التعديلات في قاعدة البيانات وإشعار المستخدم الذي تم تعديل معلوماته بالتعديل وإرسالها له بالإيميل
	يقوم بإظهار إشعار تم حفظ التعديلات بنجاح
يظهر إشعار تم حفظ التعديلات بنجاح	
التدفق البديل :	- إنتهاء مدة الجلسة يطلب من المستخدم إعادة تسجيل الدخول ثم يقوم بحفظ التعديلات. - في حال حدوث فشل في حفظ معلومات المستخدم المضاف يقوم بإظهار إشعار الرجاء إعادة المحاولة مرة أخرى. - في حال حدوث خطأ أكثر من مرة يقوم بإظهار إشعار بمعلومات الخطأ للمستخدم. - في حال قام المستخدم بالرجوع للخلف أثناء التعديل يظهر رسالة لن يتم حفظ التعديلات للمستخدم.

التدفق اللاحق :	- يتابع مدة الجلسة الزمنية ويقوم بإنهاء الجلسة تلقائيا بعد فترة محددة من عدم النشاط على جهاز الكمبيوتر لضمان الأمان ومكافحة الغش. - يقوم بحفظ وتخزين المعلومات في قاعدة البيانات وتحديث المعلومات لدى المستخدمين والفرق. -يقوم بعرض معلومات المستخدم المحدد للمدير.
-----------------	--

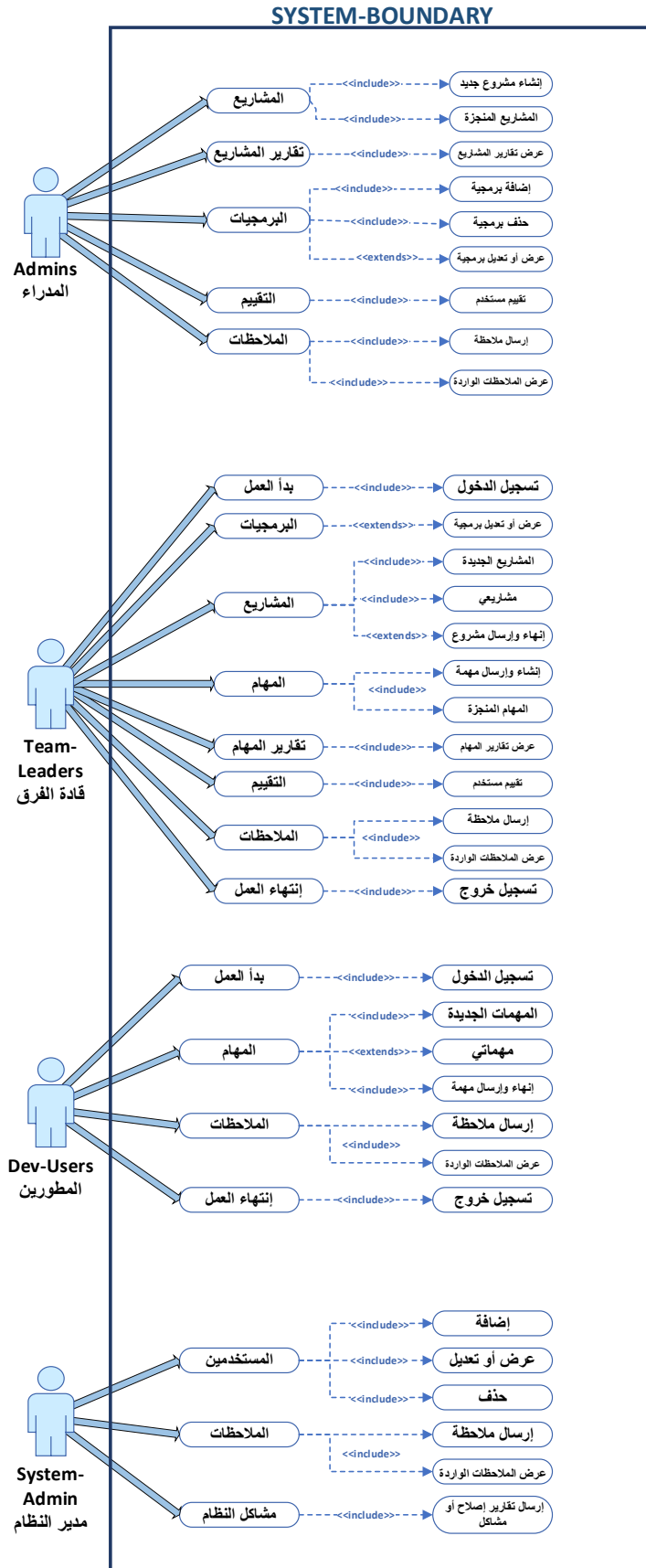
Admins المدراء

رقم حالة الإستخدام :	7
اسم حالة الإستخدام :	إنشاء مشروع جديد
الفاعل الرئيسي :	المدراء Admins
الشروط المسبقة :	- تم تسجيل الدخول بنجاح للنظام - أن يكون الحساب (مدير admin) - تم استلام مشروع من عميل
تسلسل العمليات	
المستخدم	النظام
يقوم بالنقر على قسم المشاريع في القائمة المنسدلة	
	يقوم بتوسيع خيارات جديدة : إنشاء مشروع , عرض المشاريع المنجزة , تقارير المشاريع
	يتم توسيع خيارات جديدة : إنشاء مشروع , عرض المشاريع المنجزة , تقارير المشاريع
	يقوم بالنقر على إنشاء مشروع جديد
	يتم تحويل المستخدم لواجهة إنشاء مشروع
	يتم التحويل لواجهة إنشاء مشروع
	يقوم بإدخال تفاصيل المشروع ومتطلباته وتحديد المستخدمين والفرق المنقذة له ثم ينقر على إنشاء
	يقوم بحفظ المشروع والمعلومات في قاعدة البيانات ثم إرسالها للمستخدمين والفرق المحددة
	يقوم بإشعار المستخدم تم إنشاء وإرسال المشروع بنجاح
	يظهر إشعار تم إنشاء وإرسال المشروع بنجاح

- إنتهاء مدة الجلسة يطلب من المستخدم إعادة تسجيل الدخول ثم يقوم بإرساله. - في حال حدوث فشل في حفظ معلومات المستخدم المضاف يقوم بإظهار إشعار الرجاء إعادة المحاولة مرة أخرى. - في حال حدوث خطأ أكثر من مرة يقوم بإظهار إشعار بمعلومات الخطأ للمستخدم.	التدفق البديل :
- يتابع مدة الجلسة الزمنية ويقوم بإنهاء الجلسة تلقائياً بعد فترة محددة من عدم النشاط على جهاز الكمبيوتر لضمان الأمان. - يقوم بإنشاء المشروع وحفظه في قاعدة البيانات ثم إرساله للمستخدمين والفرق المحددين. - يقوم بحفظ وتخزين المعلومات في قاعدة البيانات وتحديث المعلومات لدى المستخدمين والفرق.	التدفق اللاحق :

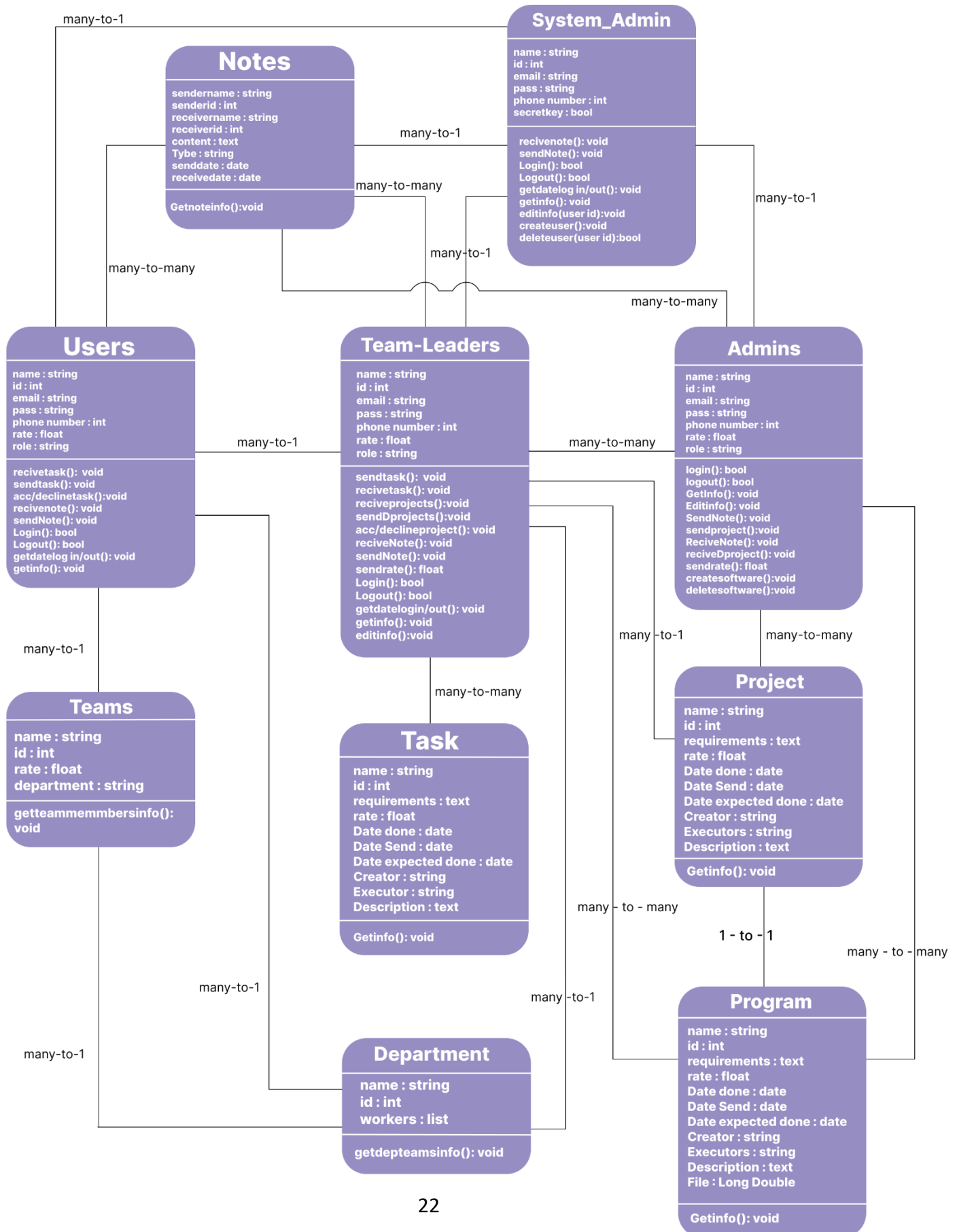
USE CASE DIAGRAM

مخطط حالات الاستخدام

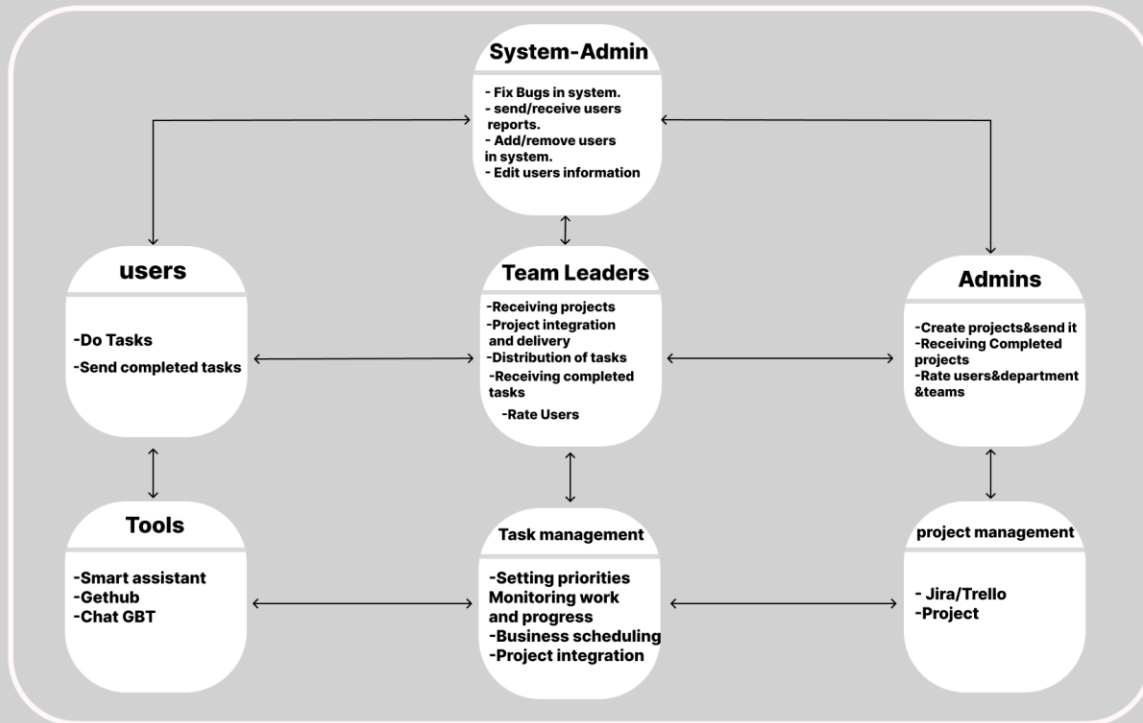


CLASS DIAGRAM

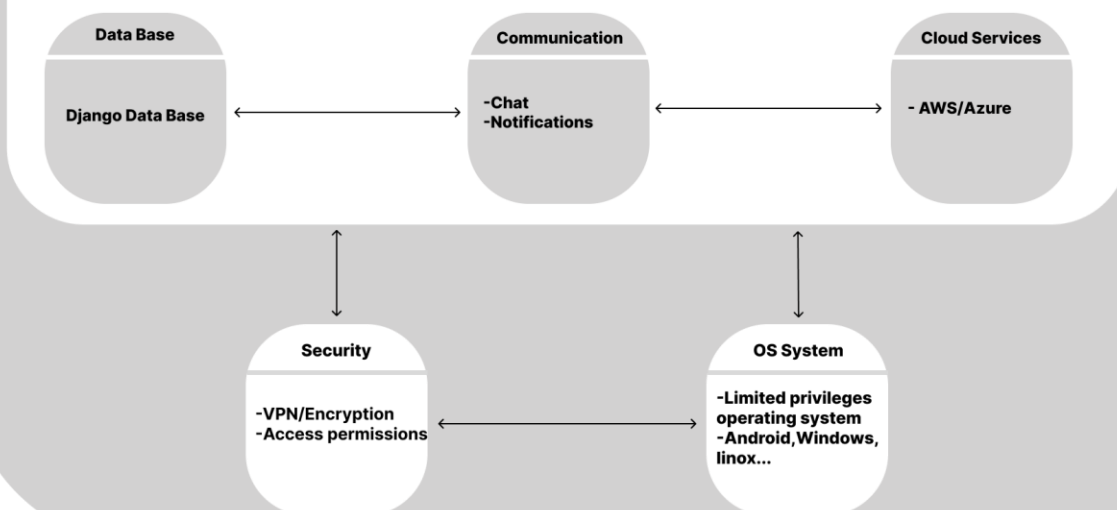
مخطط الصفوف



Online Management System نظام إدارة عن بعد



Core Infrastructure البنية التحتية الأساسية



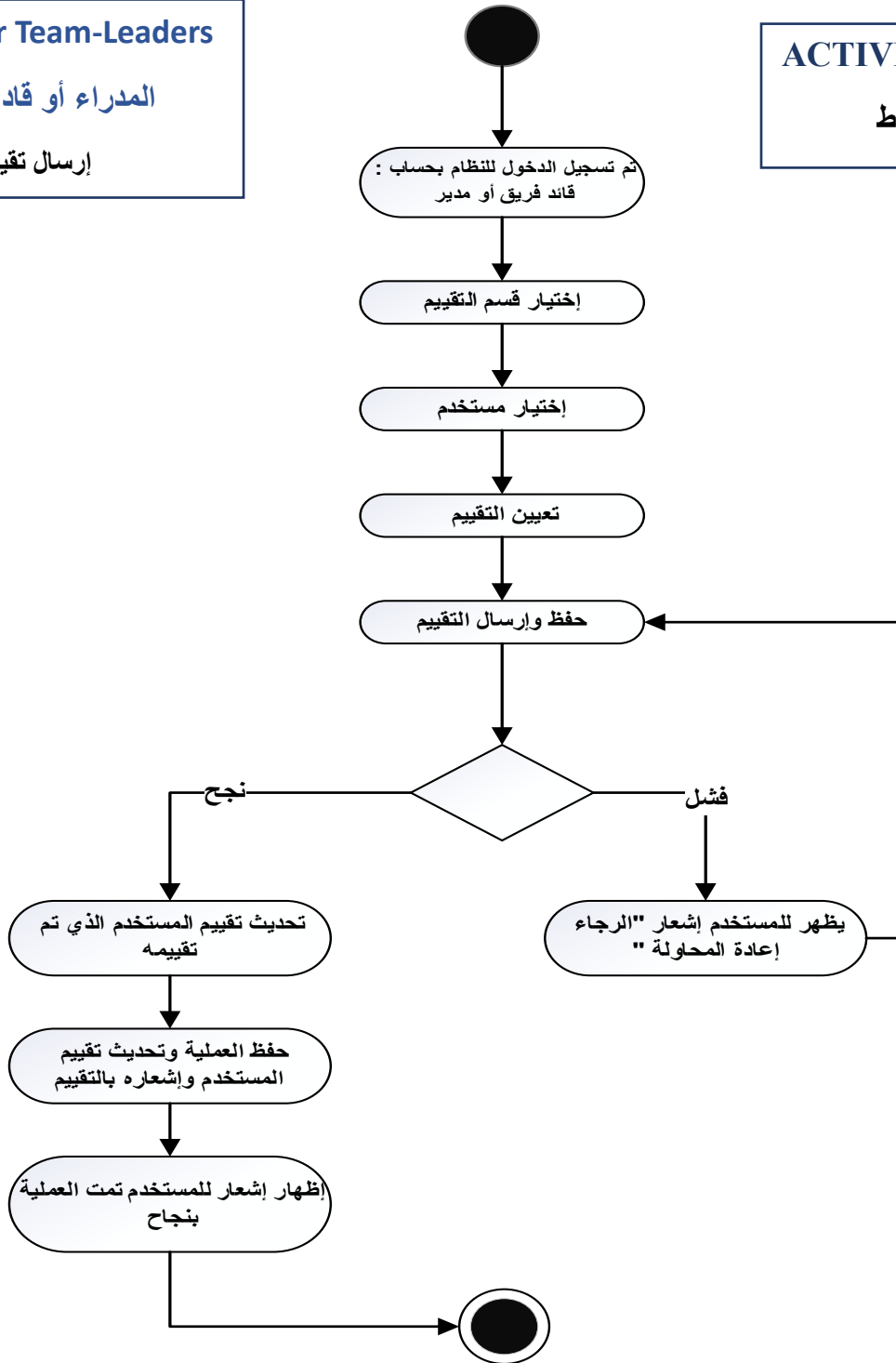
Admins or Team-Leaders

المدراء أو قادة الفرق

إرسال تقييم

ACTIVITY DIAGRAMS

مخططات النشاط

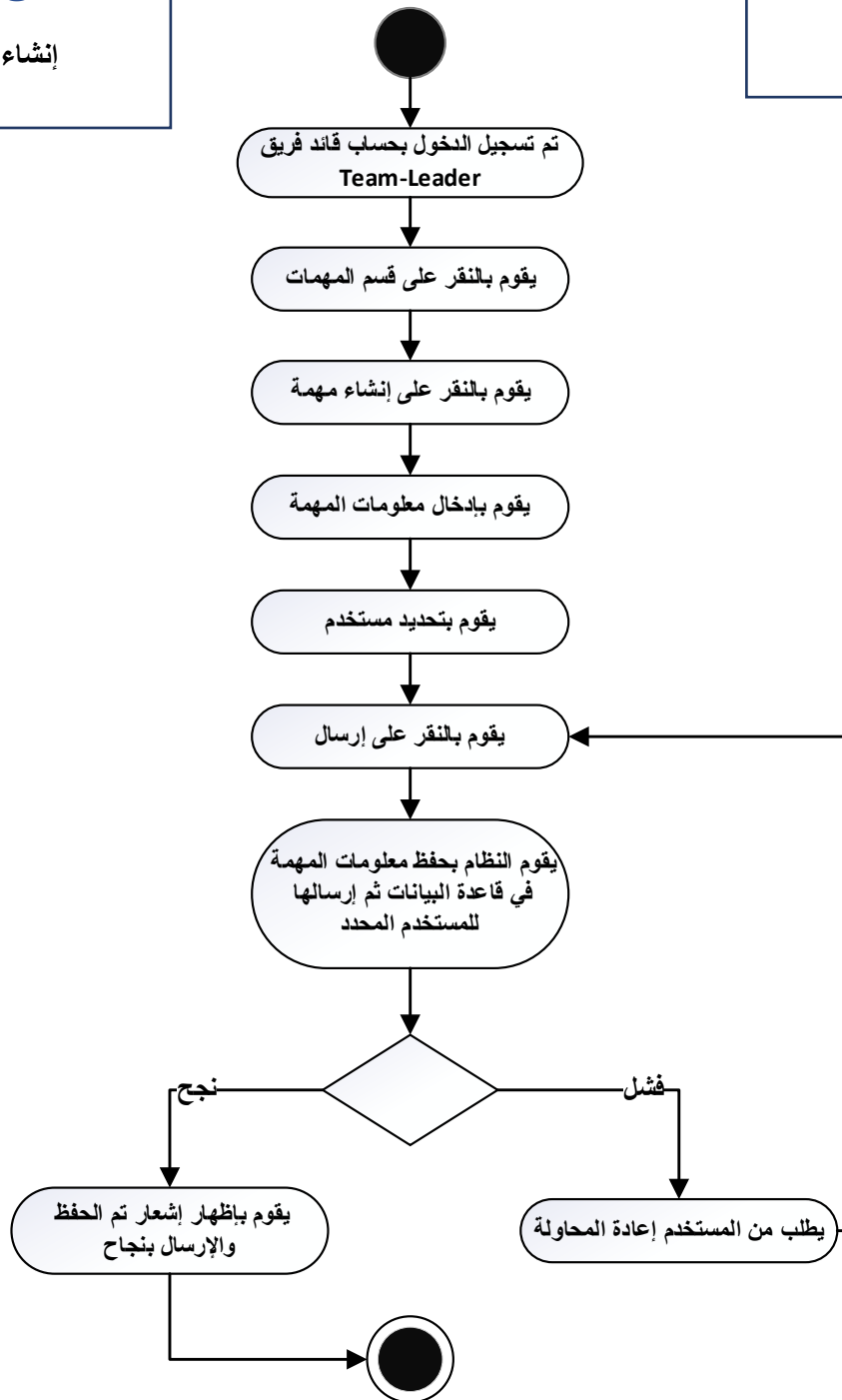


Team-Leaders قادة الفرق

إنشاء مهمة وإرسالها

ACTIVITY DIAGRAMS

مخططات النشاط



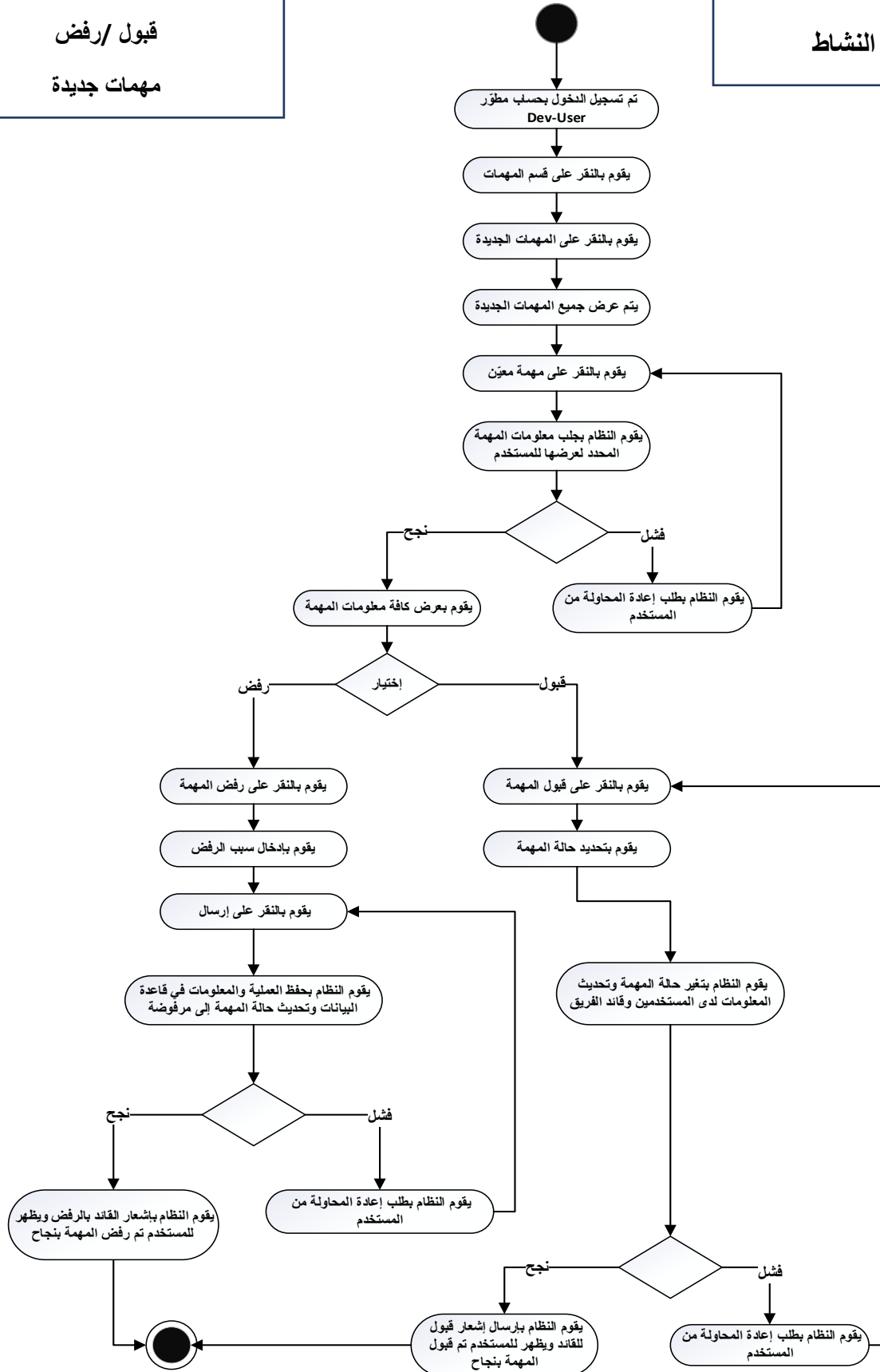
المطورين Dev-Users

قبول / رفض

مهام جديدة

ACTIVITY DIAGRAMS

مخططات النشاط

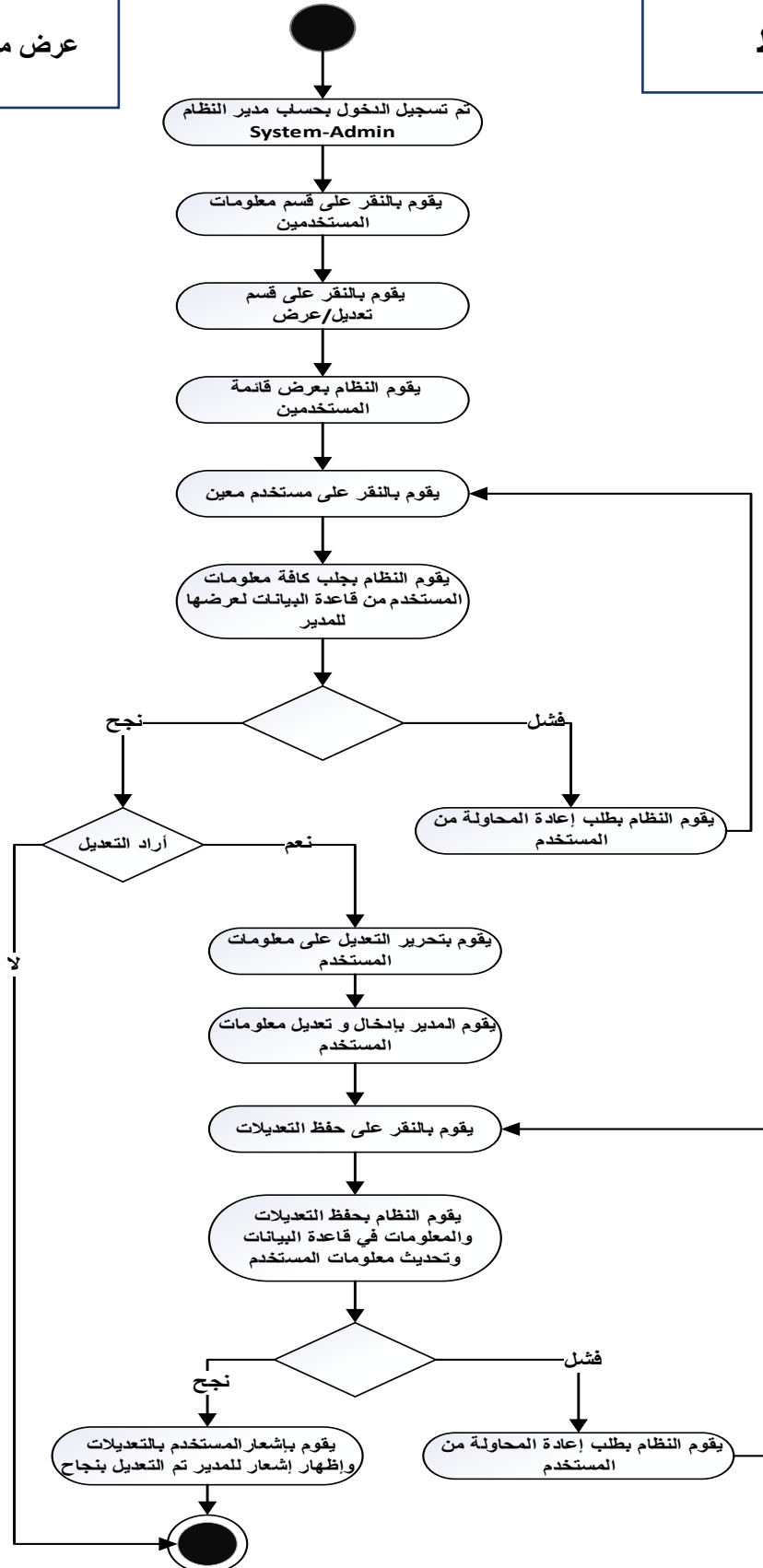


مدير النظام System-Admin

عرض معلومات مستخدم أو تعديلها

ACTIVITY DIAGRAMS

مخططات النشاط

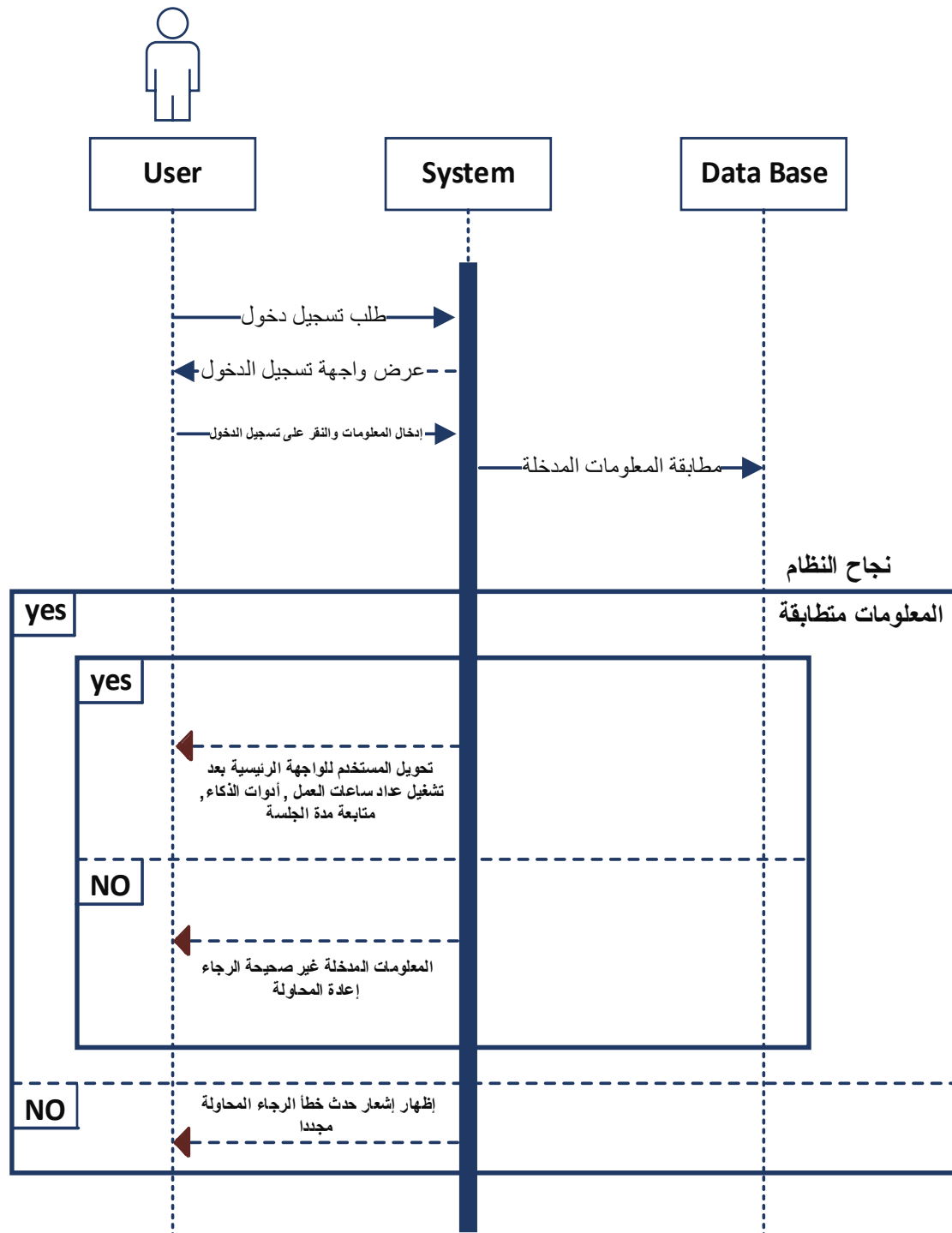


ALL USERS المستخدمين

تسجيل دخول

SEQUENCE DIAGRAMS

المخططات التسلسلية

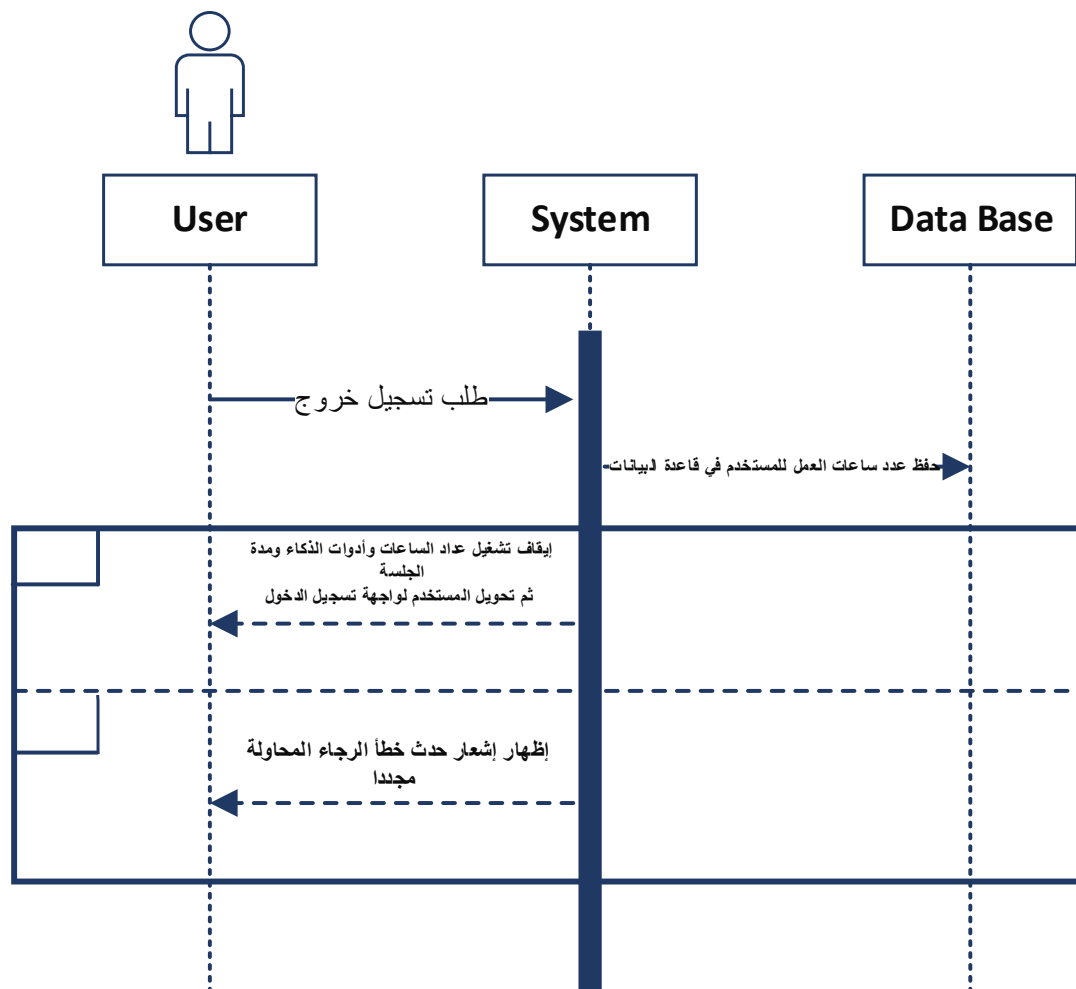


كل المستخدمين ALL USERS

تسجيل خروج

SEQUENCE DIAGRAMS

المخططات التسلسلية

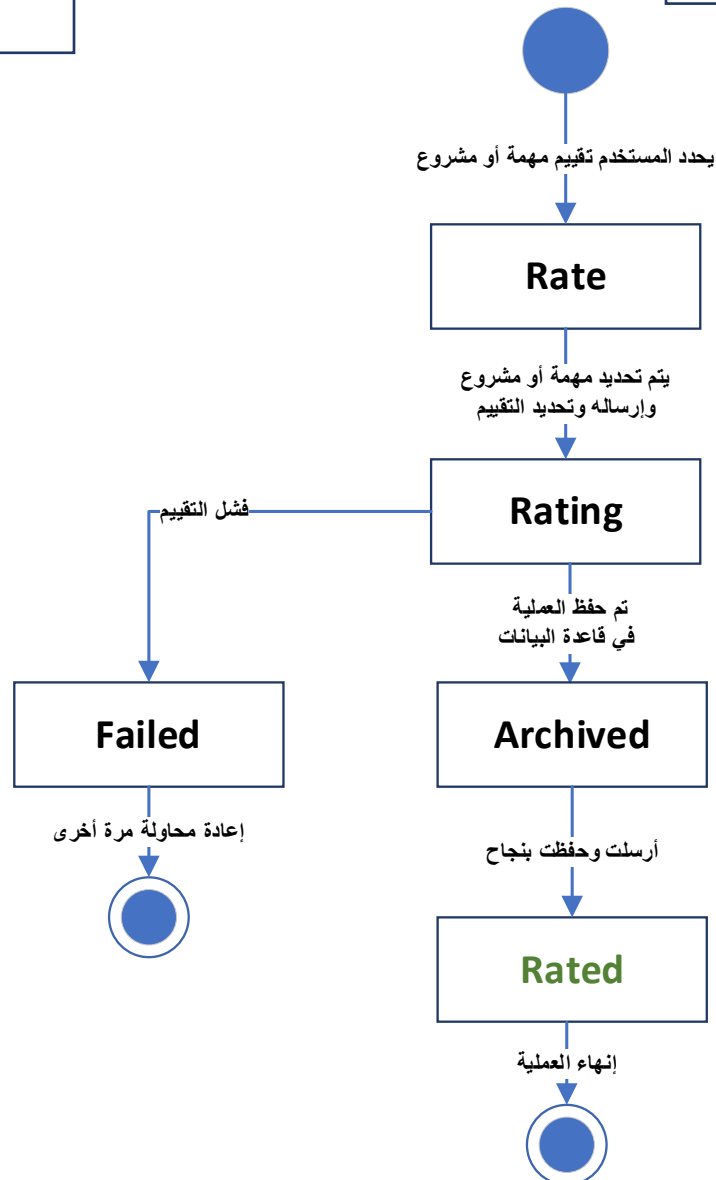


All Users

حالة الملاحظات في النظام

STATE DIAGRAMS

المخططات التسلسلية

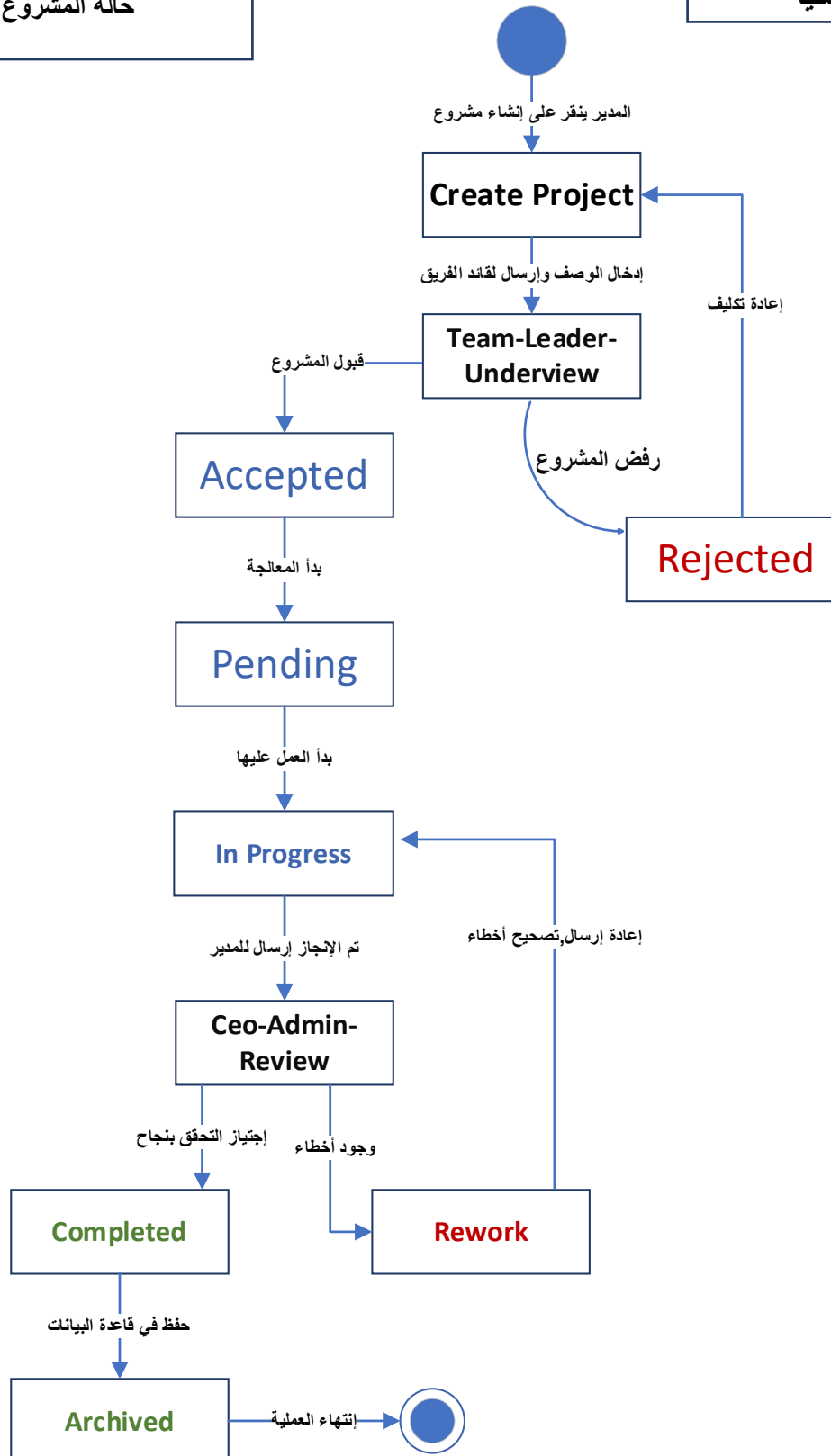


Admins + Team-Leaders

حالة المشروع في النظام

STATE DIAGRAMS

المخططات التسلسلية



المنهجيات المتبعة في تصميم النظام

قمت بدراسة تحليلية حديثة لتطوير مشروع مستقبلا وجعله أكثر مرونة وقابلا للتغيير دون الحاجة إلى إعادة هيكليّة كاملة للنظام والبدأ من جديد.

حيث قمت باتباع أهم وأحدث المنهجيات في هندسة البرمجيات وإدارة المشاريع في مشروعنا وهي :

- إدارة البرمجيات الرشيقّة (Agile Software Management)

- نماذج التصميم (Design Patterns)

-إدارة البرمجيات الرشيقّة (Agile Software Management):

باستخدام منهجية الأجايل (Agile) في الإدارة وأخذت من طرائقها الطريقة المثالية لمشروع وهي **السكرام (scrum)** وأسقطتها عليه لجعله أكثر مرونة وقابلية للتغيير.

طريقة تطبيق أسلوب الأجايل في تصميم نظام إدارة العمل عن بعد :

Agile Scrum Framework In RWMS

حيث قمت بتقسيم الأعضاء المستخدمين للنظام في الشركة معتمدة على منهج السكرام في الإدارة والعمل وتم تقسيم المستخدمين على ثلاث أعضاء أو مجموعات مختلفة كالتالي :

1. مالك المنتج Product Owner :

وفي نظامنا هم **المدراء (Admins)** بمختلف مواقعهم الوظيفي ضمن الشركة وهم صوت العميل وقمة الهرم ضمن الشركة وظائفهم كالتالي :

- استقبال العميل وفهم متطلباته المبدئية.
- إنشاء المشروع ثم تحديد الفرق التي ستقوم بتنفيذ المشروع وإرساله لهم.
- استقبال التقارير والملاحظات.
- مراقبة تقدم المشروع والعمل ضمن الشركة
- استلام المشاريع المنجزة والبرمجيات وتسليمها للعميل.
- تقييم المشروع على أساس رضى العميل وتحقيق متطلباته بالدرجة الأولى وبالدرجة الثانية اعتمادا على جودة العمل وسرعة تنفيذه.

2. القائد الخادم scrum master :

- في نظامنا هم قادة الفرق (Team-Leaders) وعقدة الربط بين فرق العمل والإدارة والمسؤول الأول عن الفريق والمشاريع ودرجة نجاحهم أو فشلهم , وظائفهم كالتالي :
- استلام المشاريع , ثم تقسيمها على أفراد الفريق حسب دراسة وخطة عمل دقيقة.
 - استلام المهام , والتأكد من صحتها وجودتها ثم مكاملتها وتنسيقها.
 - تسليم المدراء المشروع بعد مكاملة أقسامه واختبارها.
 - إدارة ومراقبة العمل ضمن الفريق , وتلبية متطلبات العمل لأفراد الفريق.
 - هم صوت المطورين للمدراء وصوت المدراء للمطورين.
 - تقييم أفراد الفريق تبعا لكفاءة عمله.
 - مراقبة تطور أفراد الفريق وإنتقاء دورهم ومهامهم حسب رؤيته لعملهم.

3. فريق التطوير Development Team :

في نظامي هم

أفراد الفرق (Dev-Users) من الأقسام الأساسية للعمل البرمجي (برمجة, تصميم, جودة, إنتاج, دعم فني, تسويق...) يعملون بشكل متكامل ومتناسق ومهامهم الرئيسية كالتالي :

- إنجاز المهام الموكلة لهم من قادة الفرق.

مدير النظام System Admin :

وهو دور فرعي ومهم لعمل نظامنا فقننا بوضعه بشكل فرعي وداعم لعمل النظام ونظرا لإختلاف مهامه ووظائفه عن باقي أفراد الشركة بشكل جذري وهو المسؤول عن كفاءة عمل النظام بكل النواحي :

- البرمجة
- إدارة الأرشفة
- إدارة قاعدة البيانات
- إدارة المستخدمين (إضافة, حذف, تعديل معلومات...)
- إصلاح أعطال النظام وصيانتة الدورية.

- قصة المستخدم User-Story :

ID	As a	I want to...	So that
1	Admin	إنشاء مشاريع جديدة في النظام	يمكنني توثيق وصف ومتطلبات العميل وتحديد الفرق وقادة الفرق المنفذة للمشروع
2		مراقبة تقدم المشاريع عبر النظام	أكتشف التأخيرات مبكراً وأتخذ إجراءات تصحيحية

3	(Product-Owner)	استلام المشاريع المكتملة من قادة الفرق	أراجعها وأسلمها للعملاء
4	Admin (Product-Owner)	تقييم المشاريع بعد التسليم	أقيس رضا العملاء وجودة الأداء لتحسين المشاريع المستقبلية
5	Team-Leader (Scrum-Master)	تقسيم المشاريع إلى مهام متفرقة	يمكنني تعيينها لأعضاء الفريق حسب الكفاءة واختصاص المطور
6		مراجعة المهام المكتملة	أضمن جودتها قبل دمجها وتسليمها للإدارة
7		إدارة احتياجات الفريق (تقنية/لوجستية)	أزيل العوائق التي تعيق إنتاجيتهم وتلبية متطلباتهم لعملية التطوير
8		تقييم أداء أعضاء الفريق دوريًا	أقوم بالتقييم بناءً على الكفاءة والأداء والجودة وسرعة التنفيذ
9	Dev-Users (Dev-Team-M)	العمل على المهام المكلف بها وتحديث حالتها	أنفذ عملي بكفاءة وأبلغ عن تقدمي
10		تسليم المهام المنجزة	تسليم المهمة بعد إنجازها واختبارها ذاتيا
11	* System-Admin	إدارة حسابات المستخدمين (إضافة/حذف/تعديل)	أضمن إضافة وتحديد الصلاحيات والمعلومات وفق التقارير
12		تتبع أعطال النظام وإصلاحها	أضمن استمرارية العمل وأقل وقت توقف النظام

- نماذج التصميم (Design Patterns):

وهي أدوات وطرائق داعمة لتصميم النظام وقمت بإتباعها في التوصيف وساقوم بتطبيقها في إعداد برمجية النظام مستقبلا , وتطبيقها على نظامي يوفر ما يلي :

- تحسين جودة وكفاءة الكود البرمجي.
- تسهيل التواصل.

- إعادة الإستخدام.
- تسهيل عملية الصيانة والتطوير.

النماذج في مشروع :

- **نموذج Singleton** : يضمن وجود نسخة واحدة فقط من فئة معيّنة من البيانات في الذاكرة, وهو فعال لتصميم الموارد المشتركة ,قمت بتطبيقه على مخططات التوصيف التي بناء عليها ساقوم بإنشاء كود نظامي البرمجي.

يوفر لي ما يلي :

- التحكم في الوصول إلى المورد المشترك (مثل ملف الإعدادات).
- توفير الذاكرة بتجنب تكرار النسخ.
- نقطة وصول موحدة للنسخة الوحيدة.

-**نموذج Adapter** : يُحوّل واجهة صنف إلى واجهة أخرى يتوقعها العميل بحيث يكون صلة الربط بين المستخدم والصنف,

الفوائد التي قدمها لي :

1. التكامل مع أنظمة خارجية : مثل GitHub&GetLab
2. إعادة استخدام الشيفرة دون الحاجة لإنشاء نسخ كثيرة.
3. عزل التغييرات : حماية الكود الداخلي من التعديلات عند تغيير واجهات خارجية.
4. توحيد البرمجية : Front+Back with API

-**نموذج Decorator** : هو نمط هيكلي يُستخدم لإضافة مسؤوليات جديدة إلى كائن ديناميكيًا دون تغيير بنيته الأصلية,

الفوائد التي قدمها لي :

إضافة خصائص ديناميكية جديدة للنظام وهي تعقب إكمال المهام وإنشاء تنبيه لها.

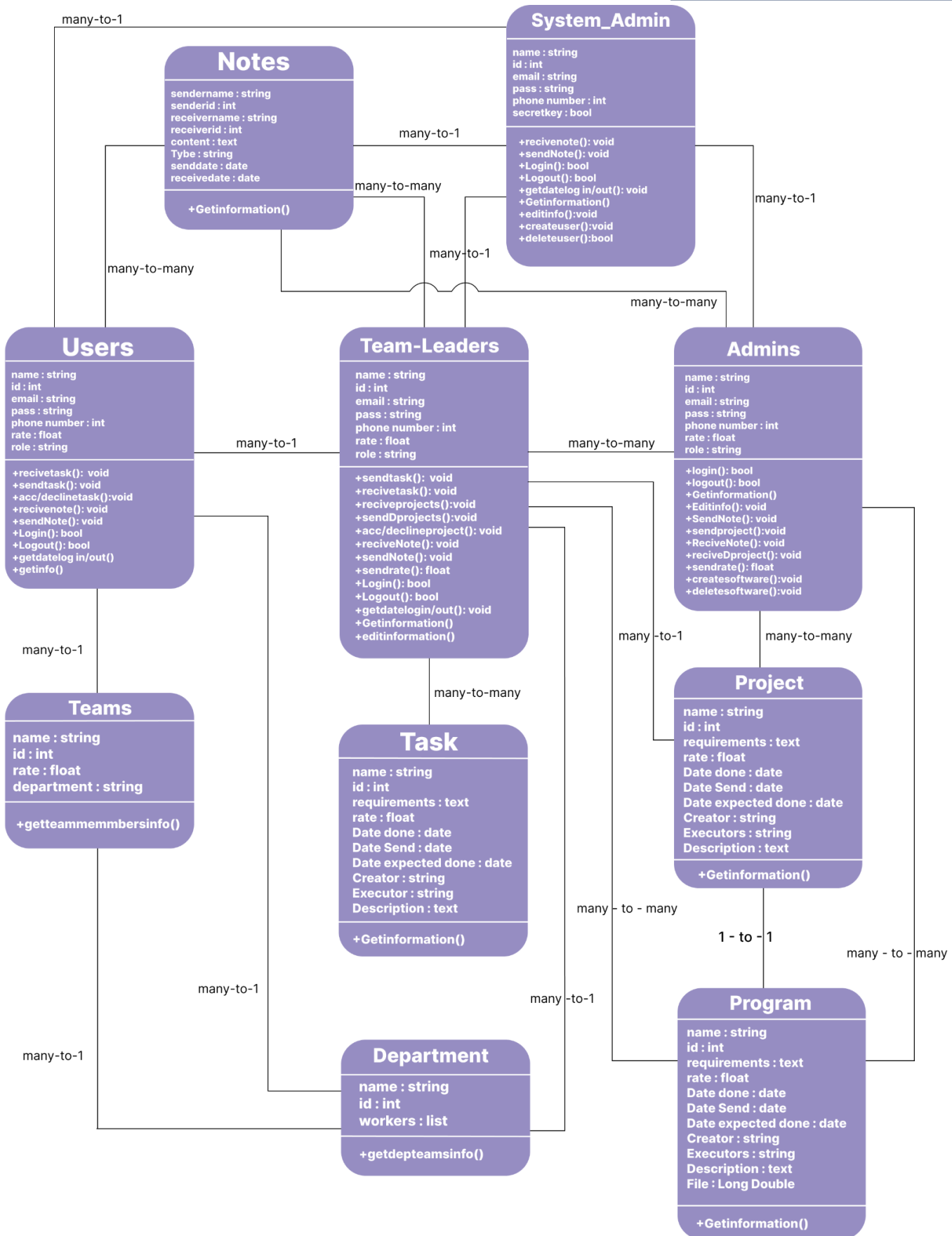
-نموذج **State** : النموذج يقوم بتصنيف أنواع المشاكل (BUGS) وتطبيق الحلول لها وتصنيفها في مرجع أو سجل وعند حدوث الخطأ يقوم بإستخدام الحل بشكل تلقائي وفي حال الفشل يعطي المستخدم المشكلة بالتفصيل للمستخدم وتصنيفها (back,front Database ,bug).

الفوائد التي قدمها لي :

1. أرشفة أنواع الأخطاء.
2. أرشفة الحلول لها وتطبيقها مباشرة عند حدوث الخطأ في حال لم يكن خطأ جديد لم يتم التعامل معه سابقا.
3. تخصيص هذه الخائص لمدير النظام وتعريفه بأنواع الأخطاء.

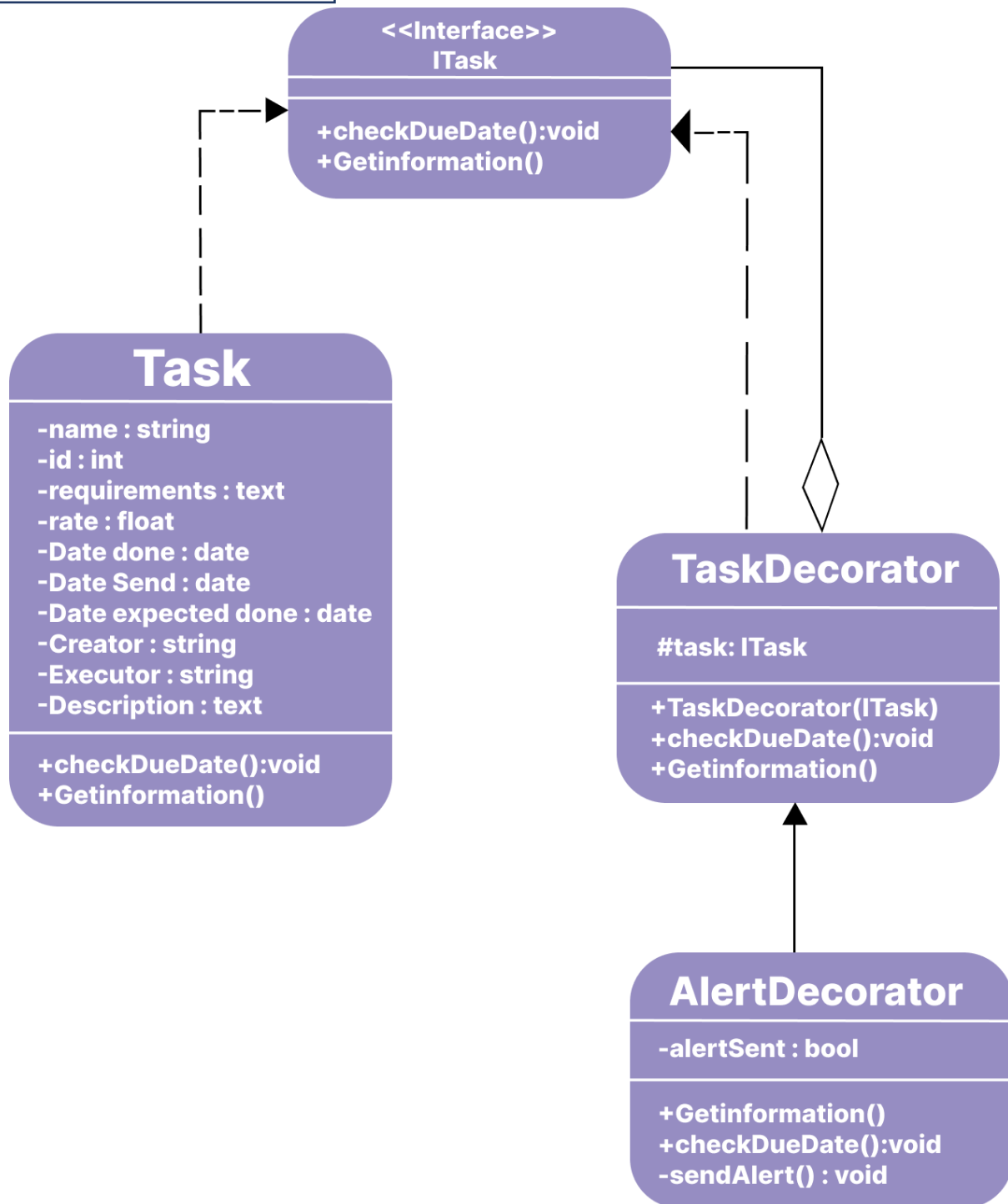
Class Diagram

Before Design Patterns



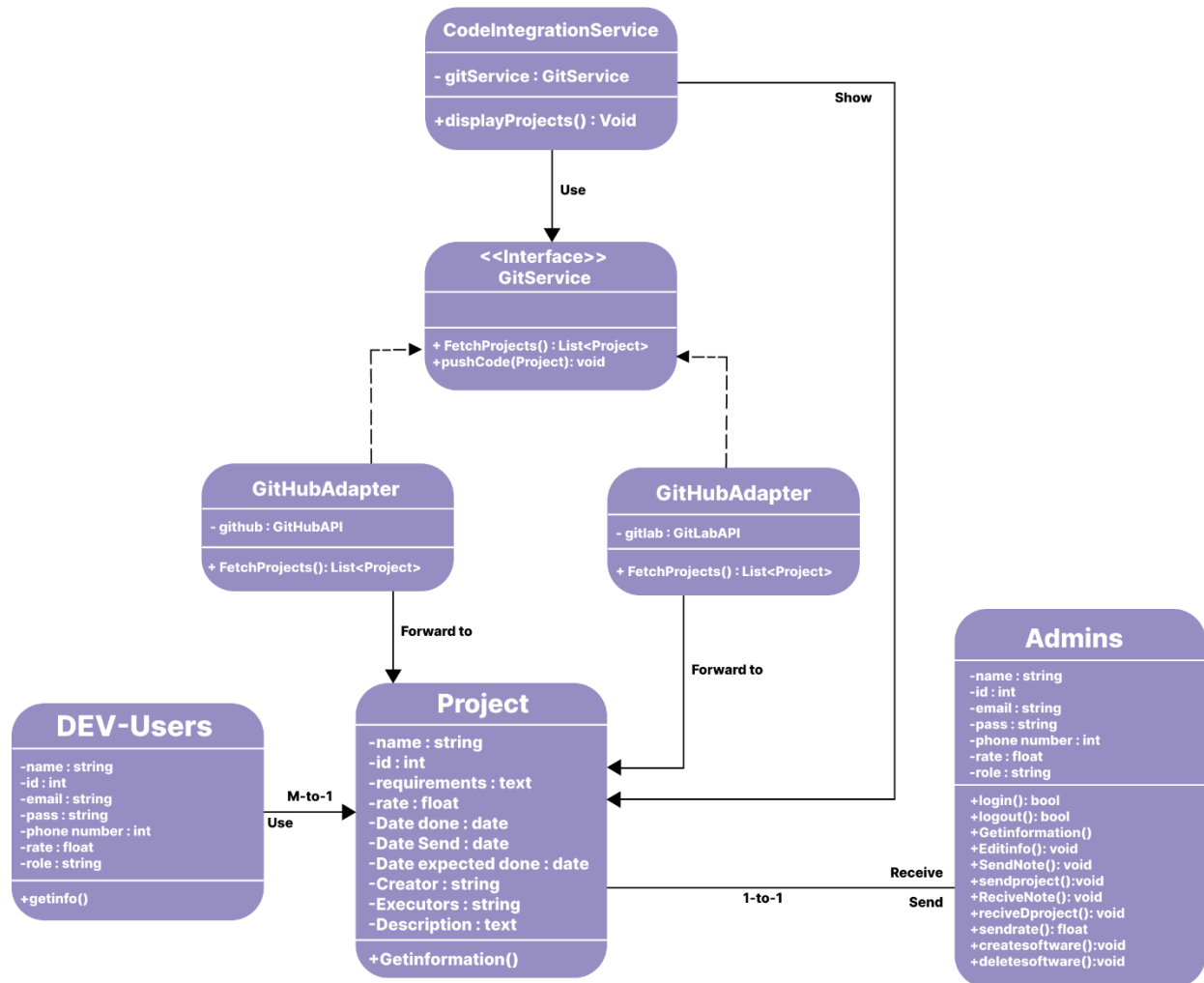
Class Diagram

After apply Dicorator



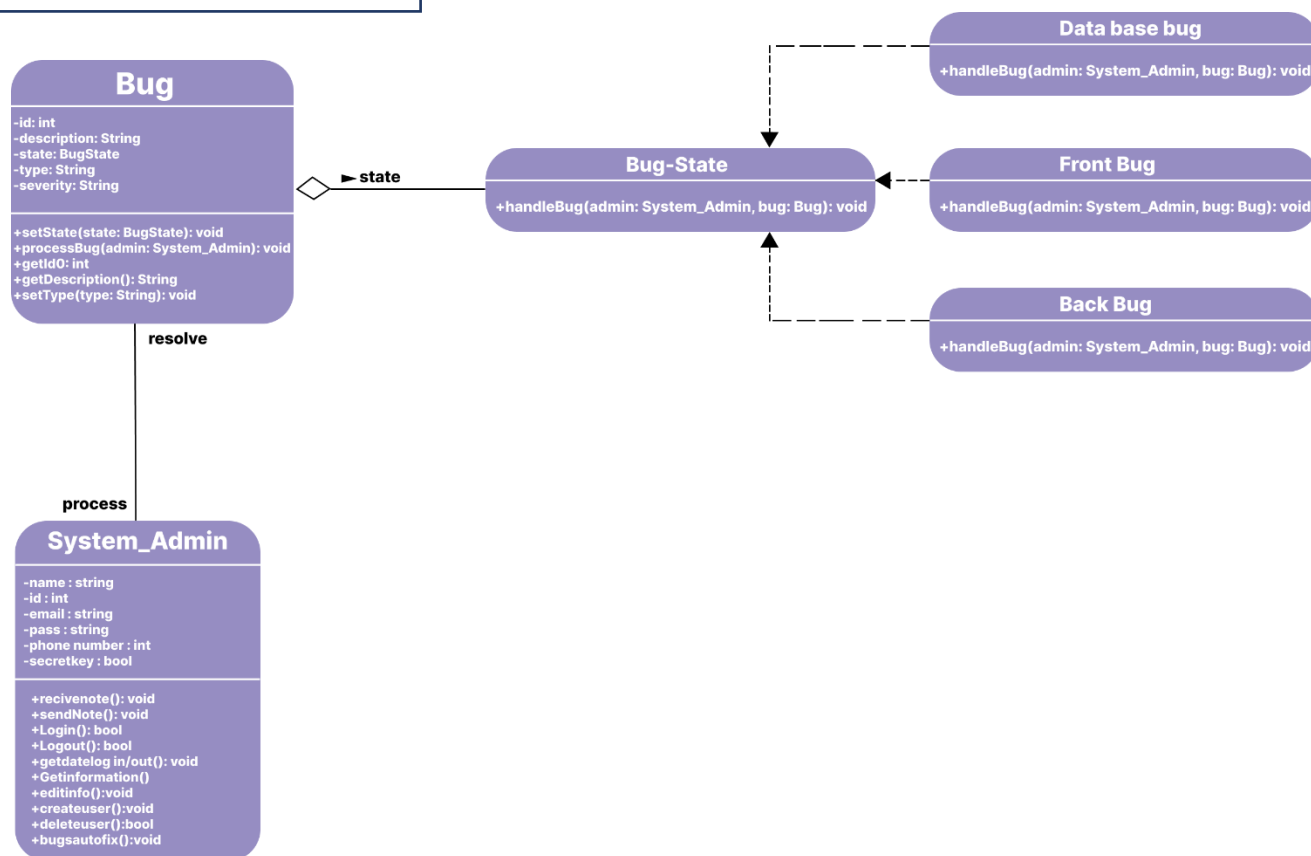
Class Diagram

After apply Adapter Pattern



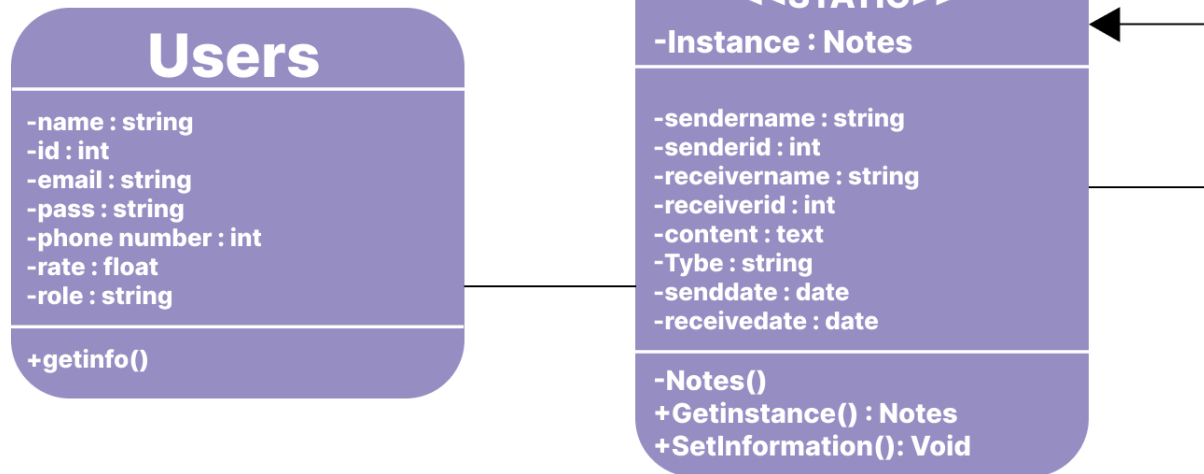
Class Diagram

After apply State Pattern



Class Diagram

After apply Singleton Pattern



Singleton Pattern

```
public class SystemConfig {
    private static SystemConfig instance;
    private SystemConfig ( ) {}
    public static SystemConfig getInstance() {
        if (instance == null) {
            instance = new SystemConfig(); }
        return instance; }}
}
```

Adapter Pattern

```
interface CodeRepository {
    String pushCode(); }

class GitHubAPI {
    public String sendCommit() {
        return "Code pushed to GitHub"; }}

class GitLabAPI {
    public String uploadProject( ) {
        return "Code pushed to GitLab"; }}

class GitHubAdapter implements CodeRepository {
    private GitHubAPI github;
    public GitHubAdapter(GitHubAPI github) {
        this.github = github; }

    @Override
    public String pushCode() {
        return github.sendCommit( ); }}

class GitLabAdapter implements CodeRepository {
    private GitLabAPI gitlab;
    public GitLabAdapter(GitLabAPI gitlab) {
        this.gitlab = gitlab; }

    @Override
    public String pushCode( ) {
```

```

        return gitlab.uploadProject( ); } }
public class Main {
    public static void main(String[] args) {
        CodeRepository repo1 = new GitHubAdapter(new GitHubAPI());
        CodeRepository repo2 = new GitLabAdapter(new GitLabAPI());
        System.out.println(repo1.pushCode( ));
        System.out.println(repo2.pushCode( )); } }

```

Decorator Pattern

```

interface Task {
    String execute( ); }
class SimpleTask implements Task {
    @Override
    public String execute( ) {
        return "Task executed"; } }
abstract class TaskDecorator implements Task {
    protected Task task;
    public TaskDecorator(Task task) {
        this.task = task; }
    public abstract String execute( ); }
class NotificationDecorator extends TaskDecorator {

```

```

public NotificationDecorator(Task task) {
    super(task); }

@Override

public String execute( ) {
    return task.execute( ) + " + Notification Sent"; } }

class TimeTrackingDecorator extends TaskDecorator {

    public TimeTrackingDecorator(Task task) {
        super(task); }

@Override

    public String execute( ) {
        return task.execute( ) + " + Time Tracked"; } }

public class Main {

    public static void main(String[] args) {
        Task task = new SimpleTask( );
        task = new NotificationDecorator(task);
        task = new TimeTrackingDecorator(task);
        System.out.println(task.execute( )); } }

```

State Pattern

```

interface BugState {
    String handle( ); }

class FrontEndBug implements BugState {

```

@Override

```
public String handle() {  
    return "Handling Front-End Bug"; }}
```

```
class BackEndBug implements BugState {
```

@Override

```
public String handle() {  
    return "Handling Back-End Bug"; }}
```

```
class DatabaseBug implements BugState {
```

@Override

```
public String handle() {  
    return "Handling Database Bug"; }}
```

```
class BugTracker {
```

```
    private BugState state;
```

```
    public BugTracker(BugState state) {
```

```
        this.state = state; }
```

```
    public void setState(BugState state) {
```

```
        this.state = state; }
```

```
    public String resolve() {
```

```
        return state.handle(); }}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        BugTracker bug = new BugTracker(new FrontEndBug());
```

```
        System.out.println(bug.resolve());
```

```
bug.setState(new DatabaseBug( ));  
System.out.println(bug.resolve( ));  
bug.setState(new BackEndBug( ));  
System.out.println(bug.resolve()); } }
```